

# Diffusion Models for Discrete Data

CS236: Deep Generative Models — Lecture 18

Seungho Go

2026-05-24

# Table of Contents

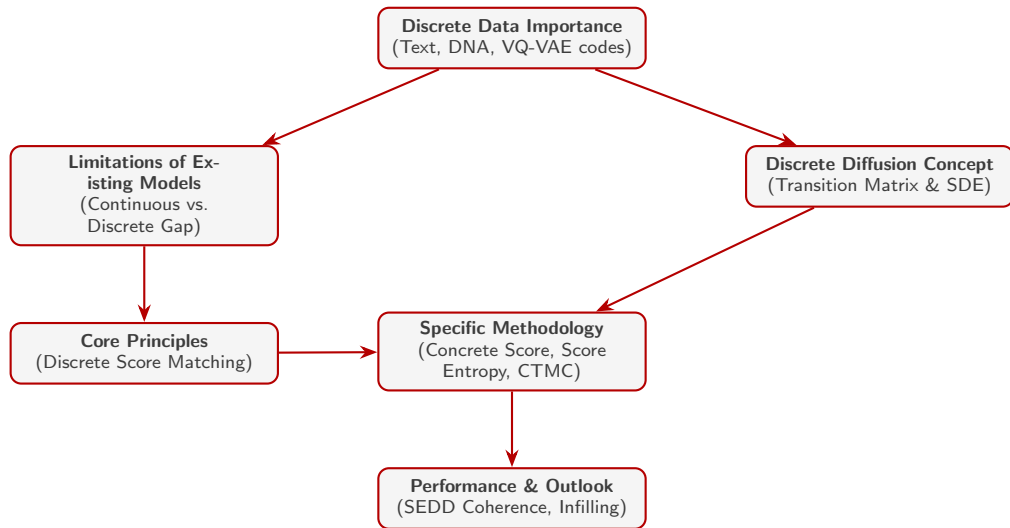
- 1 Introduction & Structure
- 2 Significance & Bottlenecks of Discrete Data
- 3 The Dominant Paradigm: Autoregressive Models
- 4 Methodology of Discrete Diffusion
- 5 Achievements & Empirical Evaluation
- 6 Conclusion & Future Outlook

# 1.1. Introduction

## Background and Motivation

- **Success in Continuous Data:** Generative models (GANs, VAEs, Continuous Diffusion) have achieved spectacular success in continuous domains like image and audio.
- **Challenges in Discrete Domains:** Innate structures of discrete data—such as natural language text, DNA sequences, and chemical structures—pose persistent challenges for gradient-based continuous models.
- **Objective:** This presentation explores the core mechanics, learning objectives (*Score Entropy*), and performance of **Discrete Space Diffusion Models** as a mathematically rigorous alternative to continuous approximations.

## 1.2. Presentation Roadmap



## 2.1. Why Discrete Data Matters

Discrete data is highly ubiquitous and represents the structural basis of various crucial domains.

### Language Data (Text)

- Formed as a non-continuous sequence of tokens.
- Pre-training LLMs is fundamentally about learning massive **discrete probability models**.

### Biological Sequences

- DNA sequences and protein amino acids are inherently discrete.
- Accurate generative modeling opens new horizons in drug discovery.

### Discretized Images

- Modern frameworks (VQ-VAE, MAGVIT-v2) quantize continuous pixel values into discrete latent spaces, showing improved generative efficiency.

## 2.2. Mathematical Bottlenecks of Generation

Traditional generative frameworks are designed for continuous spaces  $\mathbb{R}^d$  and face critical failures when directly applied to discrete spaces  $\{1, \dots, N\}^d$ .

- **Calculus Dependency:**

- **GANs and Flow-based Models** rely heavily on backpropagation gradients and variables transformation (Jacobians).
- Restricting outputs to discrete space cuts off the gradient path, making conventional training mathematically impossible.

- **Failure of Embedding & Rounding:**

- A naive strategy: embed tokens into  $\mathbb{R}^d$ , apply continuous diffusion, and round back to the nearest token during sampling.
- **Problem:** Token embedding spaces are extremely high-dimensional and sparse. The continuous model spends vast compute exploring empty space, resulting in poor sample quality and extremely slow generation (often requiring  $\sim 4,000$  steps).

## 3.1. Autoregressive (AR) Modeling Mechanics

### Sequential Factorization

AR decomposes the joint probability of a sequence  $X = (x_1, \dots, x_D)$  into causal conditional factors:

$$P(X) = \prod_{i=1}^D P(x_i \mid x_1, \dots, x_{i-1})$$

- Mimics the human process of reading and writing sequentially.
- Integrates flawlessly with **Transformer** architectures utilizing causal attention masking.

### Key Strengths

- **Scalability:** Computes the probability distribution over  $N$  tokens for only one step at a time.
- **Theoretical Capacity:** Given sufficient capacity, AR models can theoretically approximate any joint sequence distribution.
- **Natural Inductive Bias:** Pairs naturally with sequential text structures.

## 3.2. Structural Weaknesses of AR Models

Despite their massive success in NLP, AR models exhibit severe bottlenecks:

- **Sampling Drift (Error Accumulation):**
  - During generation, minor errors at early steps propagate downstream. As the sequence length increases, the model drifts away from the true data distribution.
- **Acausal / Non-sequential Data Mismatch:**
  - For biological sequences (e.g., DNA, proteins) or physical networks, there is no natural "left-to-right" causality, making AR's inductive bias forced and unnatural.
- **Sequential Generation Latency:**
  - Must generate tokens sequentially (Token-by-Token). This sequential bottleneck cannot be parallelized, leading to major inference speed constraints.



## 4.1. Generalizing Score Matching: Concrete Score

Since the continuous score  $\nabla_x \log p(x)$  is undefined in discrete domains, we use finite differences to establish the relative probability ratio.

### Mathematical Definition of Concrete Score

For a state  $x$  and any neighboring state  $y$ , the score is defined as:

$$\text{Concrete Score} \equiv \left[ \frac{p(y)}{p(x)} \right]_y$$

- **Local Neighbor Restructuring for Computational Feasibility:**

- Computing this ratio over all arbitrary states scales poorly at  $\mathcal{O}(N^{2d})$ .
- **Solution:** Restrict  $y$  to *local neighbors* differing by exactly one token. This simplifies the complexity to  $\mathcal{O}(N \cdot d)$  and allows modeling via standard Seq-to-Seq networks.

## 4.2. Learning the Concrete Score: Score Entropy

### Fundamental Score Entropy Loss

A loss function that forces a neural network  $s_\theta(x)_y$  to accurately estimate the probability ratio  $\frac{p(y)}{p(x)}$  without calculating the partition function:

$$\min_{\theta} \mathbb{E}_{x \sim p} \sum_{y \neq x} \left( s_\theta(x)_y - \frac{p(y)}{p(x)} \log s_\theta(x)_y \right)$$

- **Challenge:** The true data distribution  $p(x)$  is unknown, making the ratio  $\frac{p(y)}{p(x)}$  incomputable.
- **Solution: Denoising Score Entropy (DSE):**
  - By corrupting the clean data  $x_0 \sim p_0$  with a transition kernel  $p_t(x|x_0)$ , we bypass  $p(x)$  and derive a computable conditional objective:

$$\text{DSE}(\theta) = \mathbb{E}_{x_0 \sim p_0, x \sim p_t(\cdot|x_0)} \sum_{y \neq x} \left( s_\theta(x, t)_y - \frac{p_t(y|x_0)}{p_t(x|x_0)} \log s_\theta(x, t)_y \right)$$

## 4.3. Discrete Diffusion via Continuous Time Markov Chains

Instead of discrete SDEs, the forward noise injection is modeled as a Continuous Time Markov Chain (CTMC) over the probability vector  $p_t$ .

### Forward Transition Equation

$$\frac{dp_t}{dt} = Q_t p_t$$

Where the Transition Rate Matrix  $Q_t$  satisfies:

- ① Columns sum to 0 ( $\sum_j Q_t(j, i) = 0$ )
  - ② Non-diagonal entries are non-negative ( $Q_t(j, i) \geq 0, j \neq i$ )
- **Absorbing (Masking) Transition:** Tokens gradually transition to a designated [MASK] token.
  - **Uniform Transition:** Tokens jump randomly to any other token in the vocabulary.

## 4.4. Reverse Diffusion & Sampling Acceleration

Sampling is performed by running the Markov chain backward in time using the learned Concrete Scores.

### Reverse Transition Rate Matrix $\bar{Q}_t$

$$\bar{Q}_t(j, i) = \frac{p_t(j)}{p_t(i)} Q_t(i, j) \approx s_\theta(i, t)_j Q_t(i, j)$$

- **Discretization for High Parallelism:**

- Instead of updating one token at a time, we group multiple reverse infinitesimal transitions.
- For instance, we can unmask or flip hundreds of tokens simultaneously.
- This reduces the total required steps to a mere 64–128 steps, enabling extremely **fast parallel sampling**.

## 5.1. Coherent Generation and Speed Trade-off

The prominent discrete diffusion model—**SEDD** (Score Entropy Discrete Diffusion)—displays powerful capabilities on text generation.

- **Coherence vs. GPT-2:**
  - Outperforms or closely matches GPT-2 in generating long, semantically coherent text sequences under standard datasets (e.g., Wikitext).
- **Flexible Compute-Quality Trade-off:**
  - **Fast Sampling (64 steps):** Generates text highly rapidly, retaining comparable quality to sequential autoregressive models.
  - **High-Quality Sampling (248+ steps):** Shows a log-linear perplexity improvement as steps scale up, yielding high-fidelity text unmatched by fixed-compute models.

## 5.2. Controllable Generation via Prompt Infilling

Unlike autoregressive models that are strictly unidirectional, discrete diffusion is natively bidirectional and allows for flexible infilling.

### Prompt Infilling Mechanics

- **Bidirectional Contextualization:** Given a prefix and a suffix (or arbitrary fragments), the model fills in the missing [MASK] regions globally and simultaneously.
- **Guided Generation:** Enables conditional steering (e.g., maintaining specific motifs in DNA sequences or satisfying exact structural constraints in natural language).

## 5.3. Rigorous Likelihood Evaluation

### Mathematical Likelihood Bound

- The Denoising Score Entropy loss mathematically acts as a **likelihood upper bound** for the true sequence perplexity (PPL).
- SEDD achieves competitive or superior Perplexity compared to GPT-2 across multiple benchmark datasets, proving its strength as a density estimator.

| Model             | LAMBADA      | WikiText2    | PTB                  | WikiText103  | 1BW          |
|-------------------|--------------|--------------|----------------------|--------------|--------------|
| GPT-2-small       | <b>45.04</b> | <b>42.43</b> | 138.43               | <b>41.60</b> | <b>75.20</b> |
| SEDD-small Absorb | $\leq 52.21$ | $\leq 44.75$ | $\leq$ <b>130.49</b> | $\leq 43.14$ | $\leq 80.70$ |

**Table:** Likelihood / Perplexity (PPL) Upper Bounds comparison: GPT-2 vs. SEDD

## 6.1. Core Takeaways of Discrete Diffusion

### Paradigm Shift

- **Calculus Independence:** Liberates discrete generation from continuous constraints (rounding, Jacobian calculation, and continuous embedding exploration).
- **Rigorous Formulation:** Concrete Score and DSE provide a robust, formal, and mathematically sound probability foundation.

### Practical Value

- **No Sampling Drift:** Multi-step simultaneous denoising naturally diminishes the error accumulation found in causal sequential models.
- **Parallel Execution:** Achieves fast parallel decoding that utilizes modern GPU hardware efficiently.



## 6.2. Future Directions

Discrete space diffusion opens exciting paths across multiple frontiers:

- **Scaling to LLMs:**
  - Serving as a fast parallel-decoding alternative to standard autoregressive transformers.
- **Revolution in Bioinformatics:**
  - Tailor-made for biologically acausal sequences like DNA, RNA, and protein folds.
- **Unified Multimodal Architectures:**
  - Synthesizing quantized vision codes (VQ-VAE) and language tokens into a unified, mathematically consistent discrete diffusion pipeline.

**Thank You**